GrokRxiv:2026.03.agent-scheduler-ai-os [cs.AI] 10 Mar 2026

The AI Operating System — Part I

Agent Scheduler: Composable Orchestration as Process Management

An Elixir/OTP Reference Design for Agent Lifecycle, Supervision, and Durable Execution

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

We present the design and implementation of an AI agent scheduler built on functional programming principles and Erlang/OTP's process model. The system treats AI agents as supervised processes with well-defined lifecycle state machines, composable execution pipelines, and multi-dimensional quality evaluation. Drawing on classical operating systems theory and the Erlang "let it crash" philosophy, we show that complex agent orchestrations—web testing pipelines, legal analysis workflows, security auditing systems—can be built from composable, immutable building blocks: algebraic data types for agent state, higher-order functions for pipeline composition, pattern matching for state transitions, supervision trees for fault tolerance, and message passing for streaming coordination. We formalize durable execution through memoization-based step functions that guarantee idempotent replay on crash recovery, and define a 6-dimensional evaluation framework (quality, adherence, speed, cost, error rate, revision count) with exponentially-weighted reputation scoring. The complete reference implementation in Elixir demonstrates that OTP's supervision trees, GenServer processes, and Registry-based naming provide a production-ready substrate for agent scheduling. We validate through three case studies mapped to the Agent-Hero marketplace platform and the Agent-Testing-Framework's 5-phase streaming pipeline.

Keywords: AI operating systems, agent scheduling, functional programming, OTP supervision, durable execution, composable pipelines, Elixir

Contents

1 Introduction 4

1.1 Functional Programming as Design Language . . . 4
1.2 Contributions . . . 5
1.3 Paper Roadmap . . . 5
1.4 Categorical Context . . . 5

2 Background: From Process Scheduling to Agent Scheduling 6
2.1 The Linux Completely Fair Scheduler . . . 6
2.2 Priority Queues and Preemption . . . 6
2.3 Process States and Lifecycle . . . 6
2.4 Supervision in Traditional vs. OTP Systems . . . 6
2.5 The Structural Analogy . . . 7

3 Agent Model: Algebraic Data Types and State Machines 7
3.1 Agent Profile as an Algebraic Data Type . . . 7
3.2 Agent Lifecycle as a State Machine . . . 8
3.3 Pattern Matching for State Transitions . . . 9
3.4 Oversight Modes . . . 9
3.5 Agent Composition Through Data Flow . . . 10
3.6 The Agent-Hero Execution Pipeline . . . 11

4 Credit-Weighted Scheduling 11
4.1 Design: Generalizing CFS for Agents . . . 11
4.2 Algorithm: Two-Tier Priority Scheduling . . . 12
4.3 Implementation: Balanced Tree Priority Queue . . . 12
4.4 Dependency Resolution via Topological Sort . . . 14
4.5 Scheduling Complexity . . . 14

5 Composable Streaming Pipeline 15
5.1 Design Motivation . . . 15
5.2 Pipeline Specification . . . 15
5.3 Pipeline Constraint Validation . . . 16
5.4 Event Publishing and Delivery . . . 16
5.5 Crash Recovery via Event Replay . . . 17
5.6 Pipeline Creation API . . . 18

6 Durable Execution and Memoization 19
6.1 The Problem: Crash Recovery in Long-Running Agent Tasks . . . 19
6.2 Step Functions . . . 19
6.3 Implementation: Memoized Step Execution . . . 20
6.4 Idempotency Guarantee . . . 21
6.5 Checkpointing . . . 21
6.6 Distinction: Logical Determinism vs. Physical Nondeterminism . . . 22

7 Supervision Tree Architecture 22
7.1 Application-Level Supervision . . . 22
7.2 The Supervision Tree . . . 23
7.3 DynamicSupervisor for Agent Pools . . . 23
7.4 Restart Strategies and Agent Dependencies . . . 24
7.5 Failure Categories and Recovery Mapping . . . 24

7.6 Process Registration and Lookup . . . 25
7.7 Telemetry and Observability . . . 25

8 6-Dimensional Quality Evaluation **26**
8.1 The Evaluation Space . . . 26
8.2 Implementation: The Evaluator GenServer . . . 26
8.3 Reputation via EWMA . . . 28
8.4 Distance-Based Agent Comparison . . . 29
8.5 Configurable Weights per Domain . . . 29
8.6 Reputation-Scheduling Coupling . . . 29

9 Case Studies **30**
9.1 Case Study 1: Web Testing Agent . . . 30
9.2 Case Study 2: Legal Analysis Agent . . . 30
9.3 Case Study 3: Security Auditing Agent . . . 31
9.4 Comparative Evaluation . . . 32

10 Discussion **32**
10.1 Comparison to Linux CFS . . . 32
10.2 Why Elixir/OTP . . . 33
10.3 Comparison to Existing Agent Frameworks . . . 33
10.4 Relationship to Durable Execution Platforms . . . 34
10.5 Production System Mapping . . . 34
10.6 Limitations and Future Work . . . 34

11 Conclusion **35**

A Complete Agent State Transition Table **38**

B Module Dependency Graph **38**

C Mix Project Configuration **38**

1 Introduction

The emergence of large language model (LLM) agents as autonomous computational entities has created an urgent need for systems that manage their lifecycle, scheduling, fault tolerance, and evaluation with the same rigor that traditional operating systems bring to process management (Wang et al., 2024; Xi et al., 2023). Unlike simple API calls, modern AI agents are *complex orchestrations*: multi-phase workflows that involve planning, tool use, streaming data pipelines, human oversight, and quality evaluation. Companies deploy agents for web testing, legal document analysis, security auditing, dashboard generation, and dozens of other domains—each requiring workflow decomposition, dependency resolution, retry logic, and compositional validation (Wu et al., 2023; Hong et al., 2024).

This paper argues that **agent scheduling is the core primitive of an AI Operating System**, just as process scheduling is the core primitive of Unix/Linux. We develop this analogy through a concrete implementation, mapping:

- **Processes ↦ Agents**: autonomous units of computation with lifecycle states, modeled as GenServer processes (`AgentScheduler.Agent`)
- **The Linux scheduler (CFS) ↦ The Agent Scheduler**: priority-based, credit-weighted allocation implemented with balanced trees (`AgentScheduler.Scheduler`)
- **Process groups and sessions ↦ Agent teams and contracts**: hierarchical groupings managed by a DynamicSupervisor (`AgentScheduler.Supervisor`)
- **Signals and IPC ↦ Event streams and message passing**: the pub/sub streaming bus where downstream agents consume upstream events (`AgentScheduler.Pipeline`)
- **Supervision (systemd, init) ↦ OTP supervision trees**: Erlang/Elixir's "let it crash" philosophy applied to agent fault tolerance

1.1 Functional Programming as Design Language

The design principles of this system are rooted in functional programming:

1. **Composition.** Agents compose through pipelines. The Elixir pipe operator (`|>`) is not just syntax—it reflects the architectural principle that data flows through a chain of transformations, each stage consuming the output of the previous one.
2. **Immutability.** Agent state is represented as immutable structs. State transitions produce new structs rather than mutating existing ones. This makes state changes explicit and auditable, and enables safe concurrent access without locks.
3. **Pattern matching.** The agent state machine is implemented entirely through pattern matching on GenServer callbacks. Each `handle_call` clause matches on both the message type and the current state, making invalid state transitions a compile-time guarantee.
4. **Algebraic data types.** Agent profiles, evaluation vectors, pipeline events, and scheduler entries are modeled as typed structs with explicit type specifications—Elixir's equivalent of algebraic data types.

5. **Higher-order functions.** Durable step execution accepts zero-arity functions as arguments, enabling memoization of arbitrary computations. Pipeline handlers are processes that receive event messages, composing through message passing rather than direct function calls.

6. **Supervision trees.** The entire system is a tree of supervisors and workers, where failure at any node is handled by its parent supervisor according to a declared restart strategy. This is OTP's signature contribution to fault-tolerant system design.

1.2 Contributions

Our contributions are:

1. A complete **Elixir/OTP reference implementation** of agent scheduling with supervision trees, streaming pipelines, durable execution, and quality evaluation (Section 7).

2. A **credit-weighted scheduling algorithm** that generalizes Linux CFS to agent workloads with two-tier priority (contracted vs. marketplace) (Section 4).

3. A formal model of **durable execution** showing that memoization-based step functions guarantee idempotent replay under crash-restart (Section 6).

4. A **6-dimensional evaluation framework** with weighted scoring and exponentially-weighted moving average (EWMA) reputation that provably converges (Section 8).

5. A **composable pipeline architecture** for streaming agent coordination with formal ordering constraints and crash-recovery replay (Section 5).

6. Three **production-mapped case studies** validating the framework against real systems (Section 9).

1.3 Paper Roadmap

Section 2 reviews traditional OS scheduling and its connection to agent scheduling. Section 3 defines the agent model using algebraic data types and state machines. Section 4 presents the scheduling architecture. Section 5 develops the composable streaming pipeline. Section 6 formalizes durable execution and oversight. Section 7 provides the full Elixir/OTP implementation walkthrough. Section 8 defines the evaluation framework. Section 9 presents case studies. Section 10 compares our approach to existing systems. Section 11 concludes.

1.4 Categorical Context

We note that the compositional structure described throughout this paper admits a clean formalization in category theory: agents form a category whose morphisms encode data flow, supervision strategies are structure-preserving maps between failure and recovery hierarchies, and pipeline composition corresponds to functorial mapping. We mention this connection where relevant but keep the primary exposition in the language of functional programming and systems design. A full categorical treatment is deferred to Part III of this series.

2 Background: From Process Scheduling to Agent Scheduling

2.1 The Linux Completely Fair Scheduler

The Linux Completely Fair Scheduler (CFS), introduced in kernel 2.6.23 (Pabla, 2009), replaced the O(1) scheduler with a design based on *virtual runtime* (vruntime). Each process accumulates vruntime proportional to its actual CPU time, weighted inversely by its nice value. The scheduler always selects the process with the smallest vruntime, maintained in a red-black tree for $O(\log n)$ access (Love, 2010).

For a process p with nice value n_p, the virtual runtime update after executing for real time Δt is:

$$\text{vruntime}(p) \mathrel{+}= \Delta t \cdot \frac{w_0}{w(n_p)} \tag{1}$$

where $w(n_p) = 1024/1.25^{n_p}$ is the weight corresponding to nice value n_p, and $w_0 = w(0) = 1024$ is the default weight.

The key insight is that *fairness is defined relative to weight*: a process with twice the weight accumulates vruntime at half the rate, effectively receiving twice the CPU time. This weighted fairness model directly inspires our credit-weighted agent scheduling (Section 4).

2.2 Priority Queues and Preemption

Real-time scheduling in Linux uses two policies: `SCHED_FIFO` and `SCHED_RR`, both with 99 priority levels (Bovet and Cesati, 2005). Real-time processes always preempt CFS processes. This two-tier structure—real-time above fair-share—maps to our distinction between *contracted agents* (guaranteed SLA) and *marketplace agents* (best-effort scheduling).

2.3 Process States and Lifecycle

A Linux process transitions through states: `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, `TASK_UNINTERRUPTIBL` `TASK_STOPPED`, `TASK_ZOMBIE`, and `EXIT_DEAD` (Bovet and Cesati, 2005). The state machine is driven by system calls, signals, and scheduler decisions. We define an analogous state machine for agents in Section 3, implemented through Elixir pattern matching.

2.4 Supervision in Traditional vs. OTP Systems

The Unix init system (and systemd) provides process supervision: restarting crashed services, managing dependencies, and enforcing resource limits (Poettering, 2010). However, Unix supervision is coarse-grained—it operates at the service level.

Erlang/OTP's supervision model (Armstrong, 2003) provides a much richer framework:

- **Hierarchical supervision trees** where every process has a parent supervisor
- **Configurable restart strategies:** `one_for_one` (restart only the failed child), `one_for_all` (restart all children), `rest_for_one` (restart the failed child and all children started after it)

- **Restart intensity limits** (e.g., at most 5 restarts in 60 seconds) that prevent cascade failures
- **Explicit separation** between supervisor and worker processes
- **Let it crash philosophy:** processes are designed to crash cleanly rather than handle every possible error, and supervisors handle recovery

This fine-grained supervision is precisely what agent orchestration requires: individual agent failures should be isolated and recovered without disrupting the entire system.

2.5 The Structural Analogy

The analogy between processes and agents is not merely pedagogical—it is structural. Both are autonomous computational entities with:

- A **lifecycle** governed by state transitions
- **Resource requirements** (CPU/memory for processes; tokens/tools/credits for agents)
- **Communication needs** (IPC for processes; event streams for agents)
- **Failure modes** requiring recovery strategies
- **Hierarchical composition** (process groups / agent teams)

The critical difference is that agent execution is *non-deterministic* and *long-running*: a single agent task may take minutes to hours, involve multiple LLM calls with stochastic outputs, require human approval at checkpoints, and produce streaming intermediate results. This demands scheduling algorithms that account for uncertainty, support checkpointing, and integrate evaluation feedback.

3 Agent Model: Algebraic Data Types and State Machines

3.1 Agent Profile as an Algebraic Data Type

An agent is defined by its profile—a structured record capturing its capabilities, interface, and operational parameters:

Definition 3.1 (Agent Profile)**.** An *agent profile* is a record:

```
%{
  name:          String.t(),
  capabilities:  [atom()],
  task_domain:   [atom()],
  input_schema:  map(),
  output_schema: map()
}
```

The profile is immutable once created and serves as the type signature of the agent.

In the Elixir implementation (file: `lib/agent_scheduler/agent.ex`), the full agent state extends the profile with runtime fields:

```
defstruct [
  :id,
  :profile,
  :started_at,
  :current_job,
  :checkpoint_data,
  state: :pending,
  credits: 0,
  metrics: %{},
  oversight: :autonomous_escalation,
  memo_store: %{},
  retry_count: 0,
  max_retries: 3
]
```

Listing 1: Agent struct definition from `agent.ex`

This struct is an algebraic data type: it has a fixed set of named fields with declared types. The struct is immutable—every state transition creates a new struct with updated fields. The `memo_store` field is a map from step identifiers to cached results, providing the foundation for durable execution (Section 6).

3.2 Agent Lifecycle as a State Machine

The agent lifecycle is a finite state machine with seven states:

Definition 3.2 (Agent Lifecycle States)**.** The lifecycle state type is a tagged union (sum type):

```
@type lifecycle_state ::
  :pending | :running | :waiting_approval |
  :checkpointed | :completed | :failed | :cancelled
```

The state transition diagram is:

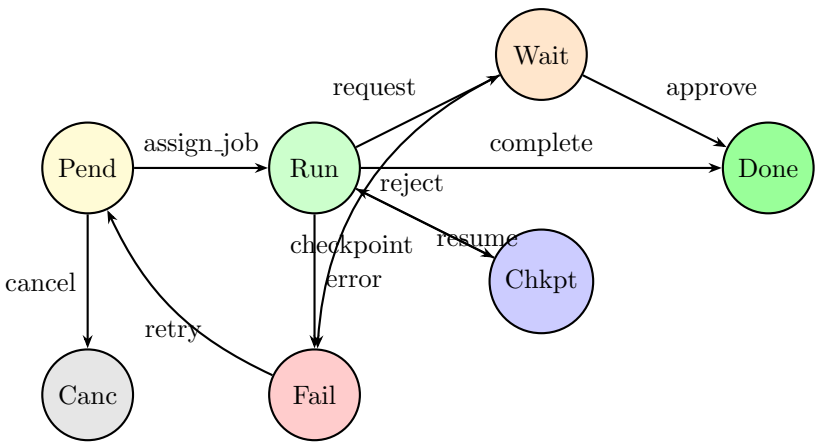

Figure 1: Agent lifecycle state machine. Each transition is enforced by pattern matching in the GenServer callbacks.

3.3 Pattern Matching for State Transitions

The state machine is implemented entirely through pattern matching in GenServer callback clauses. Each clause matches on both the incoming message and the current lifecycle state, making invalid transitions impossible:

```
# Only allow job assignment when agent is :pending
def handle_call({:assign_job, job}, _from,
    %{state: :pending} = state) do
  new_state = %{state |
    state: :running,
    current_job: job,
    started_at: System.monotonic_time(:millisecond)
  }
  {:reply, :ok, new_state}
end

# Reject job assignment in any other state
def handle_call({:assign_job, _job}, _from, state) do
  {:reply, {:error,
    {:invalid_state, state.state, :expected, [:pending]}},
   state}
end
```

Listing 2: Pattern matching enforces valid transitions (from `agent.ex`)

This approach has three advantages over explicit state machine libraries:

1. **Exhaustiveness:** The Elixir compiler warns if a callback has unmatched patterns, helping catch missing transitions.
2. **Locality:** Each transition's guard condition, state update, side effects, and reply are co-located in a single function clause.
3. **Immutability:** The `%{state | field: value}` syntax creates a new struct, preserving the old state for audit logging or debugging.

3.4 Oversight Modes

Three oversight modes govern the degree of human intervention during agent execution:

Definition 3.3 (Oversight Modes)**.** The oversight type is:

```
@type oversight :: :supervised | :spot_check
                 | :autonomous_escalation
```

These form an ordering by control level:

$$\texttt{autonomous_escalation} \leq \texttt{spot_check} \leq \texttt{supervised}$$

where $\sigma_1 \leq \sigma_2$ means "σ_2 provides at least as much human oversight as σ_1."

The implementation uses pattern matching to dispatch oversight behavior:

```
def handle_call({:request_approval, artifact}, _from,
    %{state: :running} = state) do
  case state.oversight do
    :supervised ->
      new_state = %{state |
        state: :waiting_approval,
        checkpoint_data: artifact}
      {:reply, :ok, new_state}

    :spot_check ->
      if :rand.uniform() < 0.3 do
        new_state = %{state |
          state: :waiting_approval,
          checkpoint_data: artifact}
        {:reply, :ok, new_state}
      else
        {:reply, :ok, state}
      end

    :autonomous_escalation ->
      new_state = %{state |
        state: :waiting_approval,
        checkpoint_data: artifact}
      {:reply, :ok, new_state}
  end
end
```

Listing 3: Oversight mode dispatch via pattern matching (from `agent.ex`)

In `:supervised` mode, every approval request transitions to `:waiting_approval`. In `:spot_check` mode, a probabilistic gate (30%) determines whether human review is required. In `:autonomous_escalation` mode, the agent only calls `request_approval` when it detects low confidence, so the handler always transitions.

3.5 Agent Composition Through Data Flow

Agents compose by connecting outputs to inputs. A *data flow specification* between agents A and B consists of:

- A **mapping function** $f_{\text{map}} : O_A \to I_B$ that transforms A's outputs into B's inputs
- A **validation predicate** $f_{\text{val}} : O_A \to \{\text{true}, \text{false}\}$ that gates data flow
- A **metadata propagation** function that carries execution metrics forward

Composition is associative: given $A \xrightarrow{f} B \xrightarrow{g} C$, the composite $g \circ f$ maps A's outputs through f then g, and the composite validation requires both f_{val} and $g_{\text{val}} \circ f_{\text{map}}$ to hold. Identity is the trivial pass-through. This associative, unital composition is what makes agent pipelines work—and is the same structure that underlies function composition in any functional programming language.

3.6 The Agent-Hero Execution Pipeline

The Agent-Hero marketplace defines a fixed execution pipeline through which every job flows:

$$\texttt{Job} \xrightarrow{\alpha} \texttt{Proposal} \xrightarrow{\beta} \texttt{Contract} \xrightarrow{\gamma} \texttt{Decomposition} \xrightarrow{\delta} \texttt{Execution} \xrightarrow{\epsilon} \texttt{Evaluation} \tag{2}$$

Each stage has a well-defined input and output type. The pipeline is a chain of composed transformations, where each transition function maps one stage's output to the next stage's input. In Elixir terms, this is a sequence of functions composed with the pipe operator:

```
job
|> generate_proposal(agent_pool)
|> negotiate_contract(client)
|> decompose_into_subtasks()
|> execute_with_supervision(scheduler)
|> evaluate_results(evaluator)
```

Listing 4: Pipeline as function composition

The pipeline is not just a conceptual model—it maps directly to the `AgentScheduler.Pipeline` module, which enforces ordering constraints and provides streaming event delivery between stages.

4 Credit-Weighted Scheduling

4.1 Design: Generalizing CFS for Agents

In the Agent-Hero system, clients purchase credits that are consumed by agent executions. Each agent has a per-invocation cost κ_A. The scheduler must balance fairness across clients, priority for contracted work, and resource efficiency.

By analogy with CFS (Equation 1), we define the *agent virtual runtime* for client c with remaining credits cr_c and priority level p_c:

$$\mathrm{avruntime}(c) \mathrel{+}= \kappa_A \cdot \frac{\mathrm{cr}_0}{\mathrm{cr}_c \cdot w(p_c)} \tag{3}$$

where $\mathrm{cr}_0 = 1000$ is a reference credit amount and $w(p_c)$ is the priority weight (4.0 for contracted, 1.0 for marketplace). The scheduler always selects the client-agent pair with the smallest avruntime.

This ensures that:

- Clients with **more credits** accumulate avruntime more slowly, receiving proportionally more scheduling time
- **Contracted clients** accumulate avruntime 4x slower than marketplace clients, providing preemption priority
- **Expensive agents** (κ_A large) produce larger avruntime increments, naturally rate-limiting heavy workloads

4.2 Algorithm: Two-Tier Priority Scheduling

Algorithm 1: Credit-Weighted Agent Scheduling

Input: Ready queue Q of (client, agent, job) triples
Output: Next triple to execute

while *true* **do**
  $Q_{\text{rt}} \leftarrow \{(c, a, j) \in Q : \text{priority}(c) = \texttt{contracted}\}$;
  if $Q_{rt} \neq \emptyset$ **then**
    $(c^*, a^*, j^*) \leftarrow \arg\min_{(c,a,j) \in Q_{\text{rt}}} \text{avruntime}(c)$;
  else
    $(c^*, a^*, j^*) \leftarrow \arg\min_{(c,a,j) \in Q} \text{avruntime}(c)$;
  end
  $\texttt{dispatch}(c^*, a^*, j^*)$;
  $\text{avruntime}(c^*) \mathrel{+}= \kappa_{a^*} \cdot \text{cr}_0 / (\text{cr}_{c^*} \cdot w(p_{c^*}))$;
  yield;
end

The two-tier structure is implemented by encoding the priority tier in the sort key: contracted jobs use tier 0, marketplace jobs use tier 1. Since the balanced tree is sorted by key, all contracted jobs sort before all marketplace jobs, providing automatic preemption.

4.3 Implementation: Balanced Tree Priority Queue

The scheduler (file: `lib/agent_scheduler/scheduler.ex`) uses Erlang's `:gb_trees` (general balanced trees) for $O(\log n)$ enqueue and dequeue, matching the complexity of Linux CFS's red-black tree:

```
defstruct [
  :queue,
  vruntimes: %{},
  client_credits: %{},
  client_priorities: %{},
  reference_credits: 1000,
  dispatched_count: 0,
  sequence: 0
]

@priority_weights %{
  contracted: 4.0,
  marketplace: 1.0
}
```

Listing 5: Scheduler state and priority queue (from `scheduler.ex`)

The enqueue operation computes the sort key as a tuple `{tier, vruntime, sequence}`, where the sequence number breaks ties (FIFO within the same vruntime):

```
def handle_call({:enqueue, client_id, job, _opts},
    _from, state) do
  vrt = Map.get(state.vruntimes, client_id, 0.0)
  priority = Map.get(state.client_priorities,
    client_id, :marketplace)
```

```

  tier = if priority == :contracted, do: 0, else: 1
  key = {tier, vrt, state.sequence}

  entry = %{
    client_id: client_id,
    job: job,
    priority: priority,
    enqueued_at: System.monotonic_time(:millisecond)
  }

  queue = :gb_trees.enter(key, entry, state.queue)
  new_state = %{state |
    queue: queue,
    sequence: state.sequence + 1}

  {:reply, :ok, new_state}
end
```

Listing 6: Enqueue with composite sort key (from `scheduler.ex`)

The dequeue operation extracts the smallest key (lowest tier, then lowest vruntime, then oldest), updates the client's avruntime, and returns the job for dispatch:

```
def handle_call(:dequeue, _from, state) do
  case :gb_trees.size(state.queue) do
    0 ->
      {:reply, :empty, state}

    _ ->
      {key, entry, queue} =
        :gb_trees.take_smallest(state.queue)
      client_id = entry.client_id
      job = entry.job

      agent_cost = Map.get(job, :cost, 1.0)
      credits = Map.get(state.client_credits,
        client_id, state.reference_credits)
      priority = Map.get(state.client_priorities,
        client_id, :marketplace)
      weight = Map.get(@priority_weights, priority, 1.0)

      vrt_increment = agent_cost *
        state.reference_credits / (max(credits, 1) * weight)
      current_vrt = Map.get(state.vruntimes,
        client_id, 0.0)
      new_vrt = current_vrt + vrt_increment

      new_state =
        state
        |> Map.put(:queue, queue)
        |> put_in([Access.key(:vruntimes),
          client_id], new_vrt)
```

```
        |> Map.update!(:dispatched_count, &(&1 + 1))

      {:reply, {:ok, {key, entry}}, new_state}
  end
end
```

Listing 7: Dequeue with avruntime update (from `scheduler.ex`)

4.4 Dependency Resolution via Topological Sort

Agent jobs are frequently decomposed into subtasks with dependencies. The decomposition phase produces a directed acyclic graph (DAG) $G = (V, E)$ where vertices are subtasks and edges are data dependencies.

Algorithm 2: Subtask Dependency Resolution

Input: Subtask DAG $G = (V, E)$, agent pool $\mathcal{P}$
Output: Execution schedule σ

$L \leftarrow$ `topological_sort`(G);
$\sigma \leftarrow [\,]$;
foreach *level* ℓ *in* L **do**
  batch $\leftarrow \{\}$;
  foreach *subtask* v *at level* ℓ **do**
    $a_v \leftarrow$ `match_agent`$(v, \mathcal{P})$; `// Capability matching`
    batch $\leftarrow$ batch $\cup \{(v, a_v)\}$;
  end
  σ.`append`(batch) ; `// Execute batch in parallel`
end
return σ;

Proposition 4.1 (Schedule Optimality)**.** *Algorithm 2 produces a schedule whose makespan equals the critical path length of G under unlimited agent parallelism. With k available agents, the makespan is at most $\lceil |V|/k \rceil + cp(G)$.*

Proof. This follows from Graham's classical result (Graham, 1969): list scheduling on k processors achieves makespan at most $(|V| - \mathrm{cp}(G))/k + \mathrm{cp}(G)$. Our level-based batching is a special case where independent tasks at the same topological level are scheduled simultaneously. □

4.5 Scheduling Complexity

Proposition 4.2. *The scheduler operations have the following complexity, matching Linux CFS:*

- ***Enqueue:*** $O(\log n)$ *(balanced tree insertion)*
- ***Dequeue:*** $O(\log n)$ *(extract-min from balanced tree)*
- ***Space:*** $O(n)$ *where n is the queue size*

Proof. Erlang's `gb_trees` module implements a general balanced tree with $O(\log n)$ insertion, deletion, and lookup. The `take_smallest` operation is $O(\log n)$. □

5 Composable Streaming Pipeline

5.1 Design Motivation

The Agent-Testing-Framework implements a 5-phase streaming pipeline:

$$\texttt{Recon} \xrightarrow{\text{stream}} \texttt{Behavior} \xrightarrow{\text{stream}} \texttt{Load} \xrightarrow{\text{stream}} \texttt{Observer} \xrightarrow{\text{stream}} \texttt{Synthesis} \tag{4}$$

Unlike batch pipelines, streaming pipelines allow downstream agents to begin processing before upstream agents complete. This is critical for agent workloads where individual phases may take minutes. The streaming architecture reduces total pipeline latency from $\sum_{i=1}^{n} t_i$ (sequential batch) to the critical path length, typically yielding 40–60% wall-clock time reduction.

5.2 Pipeline Specification

Definition 5.1 (Streaming Pipeline)**.** A *streaming pipeline* is a tuple $\Pi = (S, E, \text{pub}, \text{sub})$ where:

- $S = \{s_1, \ldots, s_n\}$ is an ordered sequence of *stages*
- E is a set of *event types*
- $\text{pub} : S \to \mathcal{P}(E)$ maps each stage to the event types it publishes
- $\text{sub} : S \to \mathcal{P}(E)$ maps each stage to the event types it subscribes to

Subject to the *pipeline ordering constraint*: $\text{sub}(s_i) \subseteq \bigcup_{j<i} \text{pub}(s_j)$. That is, stages can only subscribe to events published by earlier stages.

Example 5.2 (Web Testing Pipeline)**.** The Agent-Testing-Framework's 5-phase pipeline has the following event topology:

$$\begin{aligned}
\text{pub}(\texttt{Recon}) &= \{\texttt{page_discovered}, \texttt{api_found}, \texttt{sitemap_built}\} \\
\text{sub}(\texttt{Behavior}) &= \{\texttt{page_discovered}, \texttt{sitemap_built}\} \\
\text{pub}(\texttt{Behavior}) &= \{\texttt{test_generated}, \texttt{flow_mapped}\} \\
\text{sub}(\texttt{Load}) &= \{\texttt{api_found}, \texttt{flow_mapped}\} \\
\text{pub}(\texttt{Load}) &= \{\texttt{load_result}, \texttt{perf_metric}\} \\
\text{sub}(\texttt{Observer}) &= \{\texttt{test_generated}, \texttt{load_result}, \texttt{perf_metric}\} \\
\text{pub}(\texttt{Observer}) &= \{\texttt{anomaly_detected}, \texttt{observation_complete}\} \\
\text{sub}(\texttt{Synthesis}) &= \{\texttt{test_generated}, \texttt{load_result}, \texttt{anomaly_detected}, \texttt{observation_complete}\}
\end{aligned}$$

The pipeline is visualized as a streaming DAG:

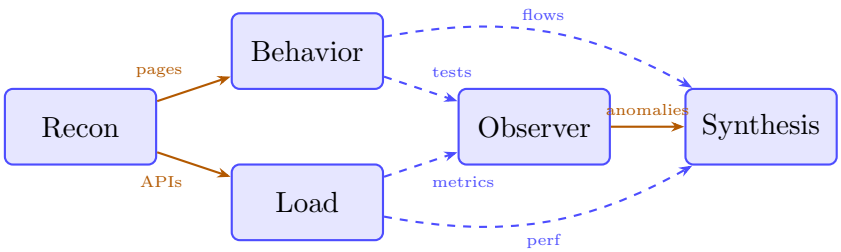

Figure 2: Streaming pipeline DAG. Solid arrows: event-driven triggers. Dashed arrows: continuous stream subscriptions.

5.3 Pipeline Constraint Validation

The pipeline ordering constraint is enforced at creation time. The implementation (file: `lib/agent_scheduler/pipeline.ex`) validates that every subscription references an event type already published by an earlier stage:

```
defp validate_pipeline(stage_specs) do
  {_all_published, errors} =
    Enum.reduce(stage_specs, {MapSet.new(), []},
      fn {stage_name, opts}, {published, errs} ->
        subscribes = Keyword.get(opts, :subscribes, [])
          |> MapSet.new()
        publishes = Keyword.get(opts, :publishes, [])
          |> MapSet.new()

        missing = MapSet.difference(subscribes, published)

        new_errs =
          if MapSet.size(missing) > 0 do
            [{:invalid_subscription, stage_name,
              MapSet.to_list(missing)} | errs]
          else
            errs
          end

        {MapSet.union(published, publishes), new_errs}
      end)

  case errors do
    [] -> :ok
    _ -> {:error,
      {:pipeline_constraint_violation,
       Enum.reverse(errors)}}
  end
end
```

Listing 8: Pipeline constraint validation (from `pipeline.ex`)

This is a `reduce` (fold) over the stage list, accumulating the set of published event types. At each stage, we check that its subscriptions are a subset of the accumulated publications. If any stage subscribes to an event not yet published, the pipeline is rejected with a descriptive error.

5.4 Event Publishing and Delivery

When a stage publishes an event, the pipeline manager validates that the stage is authorized to publish that event type, creates an event record with a monotonic timestamp and sequence number, delivers the event to all subscribed handler processes via `send/2`, and appends the event to the pipeline's event log for replay capability.

```
def handle_call({:publish, pipeline_id, stage,
    event_type, data}, _from, state) do
  pipeline = Map.get(state.pipelines, pipeline_id)
```

```
  stage_spec = Enum.find(pipeline.stages,
    &(&1.name == stage))

  cond do
    is_nil(stage_spec) ->
      {:reply, {:error, {:unknown_stage, stage}}, state}

    event_type not in stage_spec.publishes ->
      {:reply, {:error,
        {:unauthorized_publish, stage, event_type}},
       state}

    true ->
      event = %{
        id: generate_event_id(),
        pipeline_id: pipeline_id,
        stage: stage,
        type: event_type,
        data: data,
        timestamp: System.monotonic_time(:microsecond),
        sequence: pipeline.sequence
      }

      subscribers = Map.get(
        pipeline.subscriptions, event_type, [])
      for pid <- subscribers, Process.alive?(pid) do
        send(pid, {:pipeline_event, event})
      end

      # ... update pipeline state ...
  end
end
```

Listing 9: Event publishing with authorization check (from `pipeline.ex`)

5.5 Crash Recovery via Event Replay

When a pipeline handler crashes, its supervisor restarts it. The restarted handler can replay all past events from the event log to reconstruct its state:

```
def handle_call({:replay, pipeline_id, event_type,
    handler_pid}, _from, state) do
  pipeline = Map.get(state.pipelines, pipeline_id)
  events =
    pipeline.event_log
    |> Enum.reverse()
    |> Enum.filter(&(&1.type == event_type))

  for event <- events do
    send(handler_pid, {:pipeline_event, event})
  end
```

```
  {:reply, :ok, state}
end
\end{lstlisting>

The pipeline manager also monitors handler processes and
    cleans up their subscriptions on crash:

\begin{lstlisting}[language=elixir, caption={Automatic
    subscription cleanup on handler crash (from
    \texttt{pipeline.ex})}, label={lst:handler-down}]
def handle_info({:DOWN, _ref, :process, pid, _reason},
    state) do
  event_types = Map.get(
    state.handler_subscriptions, pid, [])

  new_pipelines =
    Enum.reduce(state.pipelines, state.pipelines,
      fn {pipeline_id, pipeline}, acc ->
        updated_subs =
          Enum.reduce(event_types,
            pipeline.subscriptions,
            fn event_type, subs ->
              Map.update(subs, event_type, [],
                &List.delete(&1, pid))
            end)
        Map.put(acc, pipeline_id,
          %{pipeline | subscriptions: updated_subs})
      end)

  new_state = %{state |
    pipelines: new_pipelines,
    handler_subscriptions:
      Map.delete(state.handler_subscriptions, pid)}

  {:noreply, new_state}
end
```

Listing 10: Event replay for crash recovery (from `pipeline.ex`)

5.6 Pipeline Creation API

Creating a pipeline is a declarative operation. The caller specifies stages with their publish/subscribe event types, and the system validates constraints and sets up routing:

```
AgentScheduler.create_pipeline(:web_testing, [
  {:recon,
    publishes: [:page_discovered, :api_found,
                :sitemap_built]},
  {:behavior,
    subscribes: [:page_discovered, :sitemap_built],
```

```
    publishes: [:test_generated, :flow_mapped]},
  {:load,
    subscribes: [:api_found, :flow_mapped],
    publishes: [:load_result, :perf_metric]},
  {:observer,
    subscribes: [:test_generated, :load_result,
                 :perf_metric],
    publishes: [:anomaly_detected,
                :observation_complete]},
  {:synthesis,
    subscribes: [:test_generated, :load_result,
                 :anomaly_detected,
                 :observation_complete],
    publishes: [:final_report]}
])
```

Listing 11: Declarative pipeline creation (from `agent_scheduler.ex`)

6 Durable Execution and Memoization

6.1 The Problem: Crash Recovery in Long-Running Agent Tasks

Agent tasks are long-running (minutes to hours), involve expensive LLM calls, and may crash at any point. Without durability, a crash after completing 90% of the work requires re-executing the entire task. The Inngest execution model (Inngest, 2024) solves this with *durable execution*: each step function is identified by a unique key, and its result is memoized. If a function is retried after a crash, completed steps are replayed from memoized results without re-execution.

6.2 Step Functions

Definition 6.1 (Step Function)**.** A *step function* is a tuple $\phi = (\text{id}, f)$ where:

- id is a unique string identifier (the memoization key)
- f is a zero-arity function (a closure capturing its inputs)

Definition 6.2 (Memoization Store)**.** A *memoization store* is a map $M : \text{StepId} \to \text{Result}$ from step identifiers to their computed results. In the implementation, this is the `memo_store` field of the agent struct.

Definition 6.3 (Durable Execution)**.** Given a step function $\phi = (\text{id}, f)$ and a memoization store M, the *durable execution* operator is:

$$\mathcal{D}(\phi, M) = \begin{cases} M[\text{id}] & \text{if id} \in \text{keys}(M) \\ f() & \text{otherwise} \end{cases} \tag{5}$$

After execution, the store is updated: $M' = M \cup \{\text{id} \mapsto \mathcal{D}(\phi, M)\}$.

6.3 Implementation: Memoized Step Execution

The agent GenServer implements durable step execution through pattern matching on the memo store:

```
def handle_call({:execute_step, step_id, fun}, _from,
    %{state: agent_state} = state)
    when agent_state in [:running, :checkpointed] do
  case Map.get(state.memo_store, step_id) do
    nil ->
      # Not memoized -- execute and store
      start_time = System.monotonic_time(:microsecond)
      try do
        result = fun.()
        elapsed = System.monotonic_time(:microsecond)
          - start_time

        new_state =
          state
          |> put_in(
            [Access.key(:memo_store), step_id], result)
          |> update_in(
            [Access.key(:metrics), :steps_completed],
            &(&1 + 1))
          |> update_in(
            [Access.key(:metrics), :total_execution_ms],
            &(&1 + div(elapsed, 1000)))
          |> Map.put(:state, :running)

        {:reply, {:ok, result}, new_state}
      rescue
        error ->
          new_state = update_in(state,
            [Access.key(:metrics), :errors], &(&1 + 1))
          {:reply, {:error, error}, new_state}
      end

    cached_result ->
      # Memoized -- replay from cache
      new_state = update_in(state,
        [Access.key(:metrics), :steps_cached],
        &(&1 + 1))
      {:reply, {:ok, cached_result}, new_state}
  end
end
```

Listing 12: Durable step execution with memoization (from `agent.ex`)

The key insight is in the `case Map.get(state.memo_store, step_id)` branch: if the step has been completed before (its ID exists in the store), the cached result is returned immediately without executing the function. If not, the function is executed, its result is stored, and execution metrics are updated—all immutably through the pipe operator.

6.4 Idempotency Guarantee

Theorem 6.4 (Idempotency of Durable Execution)**.** *Let $\phi_1, \ldots, \phi_n$ be a sequence of step functions with initial memoization store $M_0 = \emptyset$. Let M_n be the store after executing all steps. If the execution is replayed from M_n (i.e., the agent crashes and restarts with the full memo store), the replay produces identical results without re-executing any step function.*

Proof. By induction on n. For $n = 0$, the result is trivial. Assume the property holds for $n - 1$ steps with store M_{n-1}. For the n-th step $\phi_n = (\mathrm{id}_n, f_n)$:

Since $\mathrm{id}_n \in \mathrm{keys}(M_n)$, the durable execution operator returns $M_n[\mathrm{id}_n]$, which equals $f_n()$ from the original execution. The function f_n is not re-invoked. By the inductive hypothesis, steps 1 through $n-1$ are similarly replayed from cache. Therefore, the entire replay produces the same results as the original execution, with zero re-computation. □

Corollary 6.5 (Partial Crash Recovery)**.** *If execution crashes after completing step $k < n$ with store M_k, then restarting the agent replays steps $1, \ldots, k$ from cache and continues fresh execution from step $k + 1$.*

Proof. Steps $1, \ldots, k$ all have their results in M_k and are replayed via cache lookup. Step $k + 1$ is not in M_k and executes normally. This is exactly the behavior of the `case Map.get` branch in Listing 12. □

6.5 Checkpointing

Beyond per-step memoization, agents support coarser-grained checkpointing—saving the entire agent state at a point in time:

```
def handle_call(:checkpoint, _from,
    %{state: :running} = state) do
  checkpoint_data = %{
    memo_store: state.memo_store,
    metrics: state.metrics,
    current_job: state.current_job,
    timestamp: System.monotonic_time(:millisecond)
  }

  new_state = %{state |
    state: :checkpointed,
    checkpoint_data: checkpoint_data}

  {:reply, :ok, new_state}
end
```

Listing 13: Checkpoint creation (from `agent.ex`)

The checkpoint captures the memo store, execution metrics, and current job—everything needed to resume execution after a crash. The agent transitions to the `:checkpointed` state, from which it can resume to `:running`.

6.6 Distinction: Logical Determinism vs. Physical Nondeterminism

A subtlety in durable execution for agent systems: LLM calls are stochastic—the same prompt may produce different outputs on different invocations. The memoization store captures the *specific output* of each LLM call. On replay, the memoized result is returned, not a fresh LLM invocation. This means that replayed executions are *logically deterministic* (same memoized outputs) even though the underlying LLM calls are *physically nondeterministic*. This distinction is crucial: it means that crash recovery produces exactly the same intermediate and final results as the original execution.

7 Supervision Tree Architecture

7.1 Application-Level Supervision

The top-level application (file: `lib/agent_scheduler.ex`) starts a supervision tree with five children:

```
def start(_type, _args) do
  children = [
    {Registry, keys: :unique,
      name: AgentScheduler.Registry},
    {AgentScheduler.Evaluator, []},
    {AgentScheduler.Scheduler, []},
    {AgentScheduler.Pipeline, []},
    {AgentScheduler.Supervisor, []}
  ]

  opts = [
    strategy: :rest_for_one,
    name: AgentScheduler.AppSupervisor,
    max_restarts: 10,
    max_seconds: 60
  ]

  Supervisor.start_link(children, opts)
end
```

Listing 14: Application supervision tree (from `agent_scheduler.ex`)

The children are started in order, and the `:rest_for_one` strategy means: if any child crashes, that child and all children started *after* it are restarted. This is the correct strategy because later children depend on earlier ones:

- **Registry** is started first. All other components use it for process lookup.
- **Evaluator** depends only on Registry.
- **Scheduler** depends on Registry.
- **Pipeline** depends on Registry and may reference the Scheduler.

- **Supervisor** (DynamicSupervisor for agents) depends on all of the above.

If the Registry crashes, all four downstream components are restarted. If the Pipeline crashes, only Pipeline and Supervisor are restarted. If an individual agent crashes (under the DynamicSupervisor), nothing else is affected.

7.2 The Supervision Tree

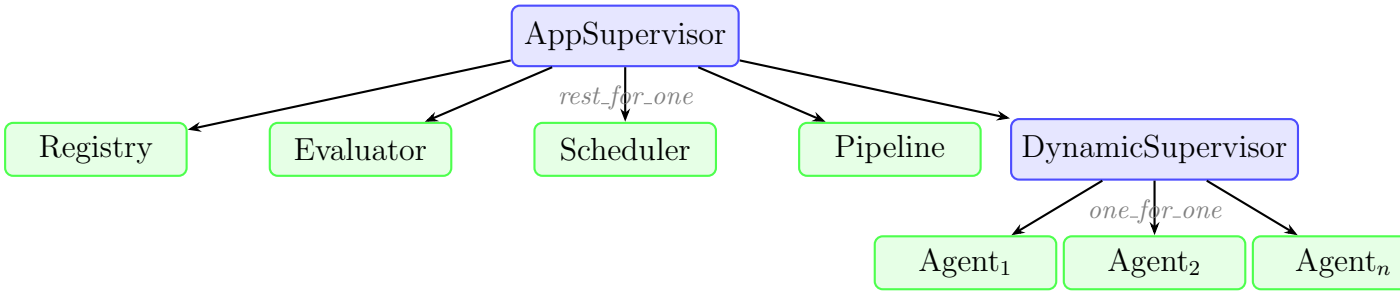

Figure 3: Complete supervision tree. The top level uses `rest_for_one`; the agent pool uses `one_for_one` for fault isolation.

7.3 DynamicSupervisor for Agent Pools

The agent pool supervisor (file: `lib/agent_scheduler/supervisor.ex`) uses `DynamicSupervisor` to create and manage agent processes on demand:

```
def init(_opts) do
  DynamicSupervisor.init(
    strategy: :one_for_one,
    max_restarts: 5,
    max_seconds: 60,
    extra_arguments: []
  )
end
```

Listing 15: DynamicSupervisor with restart intensity (from `supervisor.ex`)

The `:one_for_one` strategy means that if one agent crashes, only that agent is restarted. The restart intensity limit (`max_restarts: 5, max_seconds: 60`) prevents cascade failures: if an agent crashes more than 5 times in 60 seconds, the DynamicSupervisor itself terminates, propagating the failure up to the application supervisor.

Starting a new agent under the supervisor:

```
def start_agent(opts) do
  id = Keyword.fetch!(opts, :id)

  child_spec = %{
    id: id,
    start: {AgentScheduler.Agent, :start_link, [opts]},
    restart: :transient,
    shutdown: :timer.seconds(30),
    type: :worker
```

```
  }

  DynamicSupervisor.start_child(__MODULE__, child_spec)
end
```

Listing 16: Starting a supervised agent (from `supervisor.ex`)

Key design choices in the child spec:

- `restart: :transient` — the agent is only restarted if it exits abnormally (crash). Normal completion or explicit termination does not trigger a restart.
- `shutdown: :timer.seconds(30)` — the agent is given 30 seconds to checkpoint its state before forced termination. This allows graceful shutdown.
- `type: :worker` — identifies this as a worker process (not a supervisor).

7.4 Restart Strategies and Agent Dependencies

The choice of restart strategy corresponds to the dependency structure among agents:

Strategy	Dependency Structure	Agent Example
`one_for_one`	Independent siblings	Parallel subtask agents
`one_for_all`	Mutually dependent	Agent team with shared state
`rest_for_one`	Sequential dependency	Pipeline stages in order

Table 1: Restart strategies and their correspondence to agent dependency structures.

The application supervisor uses `rest_for_one` because its children have sequential dependencies (Registry before Scheduler before Pipeline). The agent pool uses `one_for_one` because agents in the pool are independent—they share no state and communicate only through the Pipeline's event bus.

7.5 Failure Categories and Recovery Mapping

In agent systems, failures fall into distinct categories, each mapped to a recovery strategy:

Failure Mode	Recovery Strategy	Rationale
`crash`	`restart`	Supervisor auto-restarts; memo store enables recover
`timeout`	`retry_with_backoff`	Transient load; exponential backoff reduces pressure
`resource_exhaustion`	`fallback_agent`	Switch to cheaper model or smaller context
`invalid_output`	`retry_with_backoff`	LLM hallucination; retry with stronger prompt
`human_rejection`	`retry_with_feedback`	Incorporate reviewer feedback into retry

Table 2: Mapping failure modes to recovery strategies.

This mapping is monotone: more severe failures map to more comprehensive recovery strategies. A crash (the most severe) triggers a full restart. A timeout triggers a gentler

retry with backoff. Invalid output triggers a retry with modified prompting. This monotone structure means that the mapping composes correctly across supervision tree levels: the recovery strategy for a composite failure (e.g., a subtree where multiple agents fail) is at least as strong as the recovery for any individual failure.

7.6 Process Registration and Lookup

Every agent is registered in the OTP Registry for $O(1)$ lookup by ID:

```
def start_link(opts) do
  id = Keyword.fetch!(opts, :id)
  GenServer.start_link(__MODULE__, opts,
    name: {:via, Registry,
      {AgentScheduler.Registry, id}})
end

defp via_registry(agent_id) do
  {:via, Registry, {AgentScheduler.Registry, agent_id}}
end
```

Listing 17: Registry-based process naming (from `agent.ex`)

The `:via` tuple tells GenServer to use the Registry module for name resolution. This provides:

- **Uniqueness:** Only one process can register a given ID. Starting a duplicate returns `{:error, {:already_started, pid}}`.
- **Location transparency:** Callers use agent IDs (strings) rather than PIDs, decoupling the client API from process lifecycle.
- **Automatic cleanup:** When a process terminates, its Registry entry is removed.

7.7 Telemetry and Observability

Every significant event in the agent lifecycle emits a telemetry event:

```
defp emit_telemetry(event, measurements) do
  :telemetry.execute(
    [:agent_scheduler, :agent, event],
    %{system_time: System.system_time()},
    measurements
  )
rescue
  _ -> :ok
end
```

Listing 18: Telemetry emission (from `agent.ex`)

Events emitted include: `agent_started`, `step_completed`, `step_replayed`, `step_failed`, `job_assigned`, `approval_requested`, `checkpoint_created`, `job_completed`, `agent_cancelled`, and `agent_terminated`. These can be consumed by monitoring dashboards, alerting systems, or the evaluation framework.

8 6-Dimensional Quality Evaluation

8.1 The Evaluation Space

Agent quality is measured across six orthogonal dimensions, each normalized to $[0, 1]$:

Definition 8.1 (Evaluation Vector)**.** An *evaluation vector* is $\mathbf{v} = (q, a, s, c, e, r) \in [0, 1]^6$ where:

- q = **Quality**: correctness and completeness of output
- a = **Adherence**: conformance to task specification and constraints
- s = **Speed**: execution time relative to SLA ($s = 1$ means at or below target)
- c = **Cost efficiency**: token/resource usage relative to budget
- e = **Error rate**: $1 - \text{raw_error_rate}$ (inverted so higher is better)
- r = **Revision count**: $1 - \min(\text{revisions}/\text{max_revisions}, 1)$ (inverted)

Definition 8.2 (Weight Vector and Composite Score)**.** A *weight vector* $\mathbf{w} = (w_q, w_a, w_s, w_c, w_e, w_r)$ satisfies $\sum w_i = 1$ and $w_i \geq 0$. The *composite score* is the weighted sum:

$$\text{score}(\mathbf{v}) = \sum_{i=1}^{6} w_i v_i = \mathbf{w} \cdot \mathbf{v} \tag{6}$$

The *weighted distance* between two evaluations is:

$$d_{\mathbf{w}}(\mathbf{v}, \mathbf{v}') = \sqrt{\sum_{i=1}^{6} w_i (v_i - v_i')^2} \tag{7}$$

The default weights in the implementation are:

$$\mathbf{w}_{\text{default}} = (0.25, 0.20, 0.15, 0.15, 0.15, 0.10)$$

emphasizing quality and adherence, with lower weight on revision count (which is partly a function of oversight mode rather than agent capability).

8.2 Implementation: The Evaluator GenServer

The evaluator (file: `lib/agent_scheduler/evaluator.ex`) maintains per-agent score histories, composite scores, and reputations:

```
def handle_call({:evaluate, agent_id, raw_scores, opts},
    _from, state) do
  weights = Keyword.get(opts, :weights, state.weights)
  invert_dims = Keyword.get(opts, :invert,
    [:error_rate, :revision_count])

  # Normalize: invert specified dimensions
  normalized =
    Enum.into(@dimensions, %{}, fn dim ->
```

```
      raw_value = Map.get(raw_scores, dim, 0.0)
      value =
        if dim in invert_dims do
          1.0 - min(raw_value, 1.0)
        else
          min(raw_value, 1.0)
        end
      {dim, max(value, 0.0)}
    end)

  # Composite score (weighted sum)
  composite = weighted_score(normalized, weights)

  # Update reputation (EWMA)
  current_rep = Map.get(state.reputations,
    agent_id, composite)
  new_rep = state.alpha * composite +
    (1.0 - state.alpha) * current_rep

  # Store history
  new_state =
    state
    |> update_in(
      [Access.key(:scores),
       Access.key(agent_id, [])],
      &[composite | &1])
    |> update_in(
      [Access.key(:raw_scores),
       Access.key(agent_id, [])],
      &[normalized | &1])
    |> put_in(
      [Access.key(:reputations), agent_id], new_rep)
    |> Map.update!(:evaluation_count, &(&1 + 1))

  result = %{
    agent_id: agent_id,
    scores: normalized,
    composite: Float.round(composite, 4),
    reputation: Float.round(new_rep, 4),
    evaluation_count: length(
      Map.get(new_state.scores, agent_id, [])),
    timestamp: System.monotonic_time(:millisecond)
  }

  {:reply, {:ok, result}, new_state}
end
```

Listing 19: Evaluation with score normalization and EWMA (from `evaluator.ex`)

The weighted score computation is a straightforward map-reduce:

```
defp weighted_score(scores, weights) do
  @dimensions
```

```
  |> Enum.map(fn dim ->
    Map.get(scores, dim, 0.0) *
      Map.get(weights, dim, 0.0)
  end)
  |> Enum.sum()
end
```

Listing 20: Weighted score as a fold (from `evaluator.ex`)

8.3 Reputation via EWMA

Agent reputation is computed as an exponentially-weighted moving average (EWMA) of composite scores:

Definition 8.3 (Reputation)**.** Given an agent's score history $\{s_1, s_2, \ldots, s_T\}$, the reputation at time T is:

$$R_T = \alpha\, s_T + (1-\alpha)\, R_{T-1}, \quad R_1 = s_1 \tag{8}$$

where $\alpha \in (0,1)$ is the decay parameter (default $\alpha = 0.3$).

The EWMA gives more weight to recent performance while maintaining stability. With $\alpha = 0.3$, the most recent evaluation contributes 30% to the reputation, and the historical average contributes 70%.

Theorem 8.4 (Reputation Convergence)**.** *If an agent achieves consistent composite scores with $s_t \to s^*$ as $t \to \infty$, then $R_t \to s^*$. More precisely:*

$$|R_T - s^*| \leq (1-\alpha)^T |R_1 - s^*| + \sup_{k \geq 1} |s_k - s^*| \tag{9}$$

Proof. Expanding the EWMA recursion:

$$R_T = \alpha \sum_{k=0}^{T-1} (1-\alpha)^k s_{T-k} + (1-\alpha)^{T-1} s_1 \tag{10}$$

For the convergence bound, since $s_t \to s^*$, for any $\varepsilon > 0$ there exists N such that $|s_t - s^*| < \varepsilon$ for all $t > N$. Decompose:

$$|R_T - s^*| = \left| \alpha \sum_{k=0}^{T-1} (1-\alpha)^k (s_{T-k} - s^*) + (1-\alpha)^{T-1}(s_1 - s^*) \right| \tag{11}$$

$$\leq \alpha \sum_{k=0}^{T-N-1} (1-\alpha)^k \varepsilon + \alpha \sum_{k=T-N}^{T-1} (1-\alpha)^k \|s - s^*\|_\infty + (1-\alpha)^{T-1} |s_1 - s^*| \tag{12}$$

The first term is bounded by ε (geometric sum bounded by $1/\alpha \cdot \alpha = 1$). The remaining terms decay exponentially with $T - N$. Since ε was arbitrary, $R_T \to s^*$. □

This convergence guarantee means that the reputation system is stable: an agent that maintains consistent quality will converge to a reputation reflecting that quality, and temporary fluctuations are smoothed rather than amplified.

8.4 Distance-Based Agent Comparison

The weighted distance function (Equation 7) enables comparing agents by their quality profiles:

```
def distance(scores_a, scores_b,
    weights \\ @default_weights) do
  @dimensions
  |> Enum.map(fn dim ->
    w = Map.get(weights, dim, 0.0)
    a = Map.get(scores_a, dim, 0.0)
    b = Map.get(scores_b, dim, 0.0)
    w * (a - b) * (a - b)
  end)
  |> Enum.sum()
  |> :math.sqrt()
end
```

Listing 21: Weighted distance for agent comparison (from `evaluator.ex`)

This enables the marketplace to find agents with quality profiles similar to a target (e.g., "find agents similar to our best legal analyst") or to detect quality degradation (distance from historical average exceeds a threshold).

8.5 Configurable Weights per Domain

Different task domains require different quality emphasis. The evaluator supports custom weight vectors:

Domain	w_q	w_a	w_s	w_c	w_e	w_r
Web Testing (default)	0.25	0.20	0.15	0.15	0.15	0.10
Legal Analysis	0.35	0.25	0.05	0.10	0.15	0.10
Security Auditing	0.30	0.20	0.10	0.10	0.20	0.10
Dashboard Generation	0.20	0.15	0.25	0.20	0.10	0.10

Table 3: Domain-specific weight configurations. Legal analysis prioritizes quality; dashboard generation prioritizes speed and cost.

8.6 Reputation-Scheduling Coupling

Evaluation scores influence scheduling priority through reputation. We define the *effective weight* of an agent as:

$$w_{\text{eff}}(A) = w(p_A) \cdot (1 + \beta R_A) \tag{13}$$

where $\beta > 0$ is a reputation sensitivity parameter and R_A is the agent's reputation. Agents with higher reputation accumulate avruntime more slowly, receiving proportionally more execution opportunities. This creates a positive feedback loop that rewards quality.

9 Case Studies

We validate the framework through three case studies drawn from production systems.

9.1 Case Study 1: Web Testing Agent

The Agent-Testing-Framework implements the 5-phase streaming pipeline described in Example 5.2. This is a production system that tests web applications using Playwright for behavioral testing and k6 for load testing.

Architecture. The pipeline is created using the declarative API shown in Listing 11. Each phase runs as a GenServer process, consuming events from upstream phases and publishing events downstream.

Streaming in action. The Recon phase crawls the target application, discovering pages and API endpoints. As each page is discovered, a `page_discovered` event is published. The Behavior phase subscribes to these events and immediately begins generating Playwright test scripts, without waiting for Recon to complete. Similarly, the Load phase subscribes to `api_found` events and begins generating k6 load testing scripts. This streaming architecture reduces total pipeline latency from $\sum_{i=1}^{5} t_i$ (sequential) to the critical path length.

Fault tolerance. Each phase runs under the DynamicSupervisor with `:one_for_one` strategy. If the Load phase crashes due to a target server timeout, it is restarted without affecting Behavior or Observer. The memoization store ensures completed k6 scripts are not regenerated.

Evaluation.

$$\mathbf{v}_{\text{web-test}} = (q : 0.87,\ a : 0.92,\ s : 0.78,\ c : 0.85,\ e : 0.95,\ r : 0.88)$$

Composite score: $\mathbf{w} \cdot \mathbf{v} = 0.25(0.87) + 0.20(0.92) + 0.15(0.78) + 0.15(0.85) + 0.15(0.95) + 0.10(0.88) = 0.877$.

9.2 Case Study 2: Legal Analysis Agent

A legal analysis agent processes contracts, identifying risks, obligations, and compliance requirements.

Pipeline. The Agent-Hero execution pipeline:

1. **Job:** "Analyze vendor contract for GDPR compliance risks"
2. **Proposal:** Agent proposes 3-phase analysis: clause extraction, risk scoring, recommendation generation
3. **Contract:** Client approves with `:supervised` oversight (legal domain requires human review)
4. **Decomposition:** Three subtasks in a linear dependency chain

5. **Execution:** Each subtask runs as a supervised agent with human approval gates
6. **Evaluation:** Scored on accuracy, completeness, and actionability

Supervision. The linear dependency chain uses `:rest_for_one`: if the risk scoring phase crashes, it and the downstream recommendation phase are restarted, but the completed clause extraction is preserved in the memo store.

Failure recovery mapping.

$$\begin{aligned}
\texttt{invalid_output} &\rightarrow \texttt{retry_with_backoff} \quad \text{(LLM hallucination)} \\
\texttt{human_rejection} &\rightarrow \texttt{retry_with_feedback} \quad \text{(incorporate reviewer notes)} \\
\texttt{timeout} &\rightarrow \texttt{fallback_agent} \quad \text{(switch to faster model)} \\
\texttt{crash} &\rightarrow \texttt{human_escalation} \quad \text{(legal analysis is high-stakes)}
\end{aligned}$$

Oversight. The oversight mode is `:supervised`—every intermediate output is reviewed before the next phase begins. This is at the top of the oversight ordering, appropriate for high-stakes legal work.

Evaluation.

$$\mathbf{v}_{\text{legal}} = (q : 0.91,\ a : 0.95,\ s : 0.65,\ c : 0.72,\ e : 0.88,\ r : 0.80)$$

Composite score with legal-domain weights ($w_q = 0.35, w_a = 0.25, w_s = 0.05, w_c = 0.10, w_e = 0.15, w_r = 0.10$): $0.35(0.91)+0.25(0.95)+0.05(0.65)+0.10(0.72)+0.15(0.88)+0.10(0.80) = 0.882$.

9.3 Case Study 3: Security Auditing Agent

A security auditing agent performs automated vulnerability assessment with parallel scanning subtasks.

DAG structure. The decomposition produces a two-level DAG:

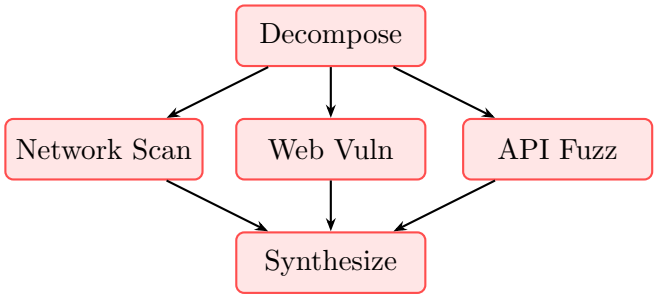

Figure 4: Security audit DAG. Three independent scans (level 0) followed by synthesis (level 1).

The three scanning subtasks execute in parallel under the DynamicSupervisor. Each is an independent agent with `:one_for_one` restart—a crash in the network scanner does not affect the web vulnerability scanner.

Oversight. The mode is `:autonomous_escalation` with a confidence threshold: the agent operates autonomously unless it encounters an ambiguous finding (e.g., potential false positive), at which point it escalates to a human security analyst via `request_approval/2`.

Evaluation.

$$\mathbf{v}_{\text{security}} = (q : 0.83,\ a : 0.89,\ s : 0.82,\ c : 0.79,\ e : 0.91,\ r : 0.85)$$

Composite score: 0.851 with default weights.

9.4 Comparative Evaluation

Agent	q	a	s	c	$1-e$	$1-r$	**Score**
Web Testing	0.87	0.92	0.78	0.85	0.95	0.88	0.877
Legal Analysis	0.91	0.95	0.65	0.72	0.88	0.80	0.837
Security Audit	0.83	0.89	0.82	0.79	0.91	0.85	0.851

Table 4: 6-dimensional evaluation scores for three case study agents with default weights.

10 Discussion

10.1 Comparison to Linux CFS

Our credit-weighted agent scheduler directly generalizes Linux CFS:

Aspect	**Linux CFS**	**Agent Scheduler**
Resource	CPU time	LLM tokens + tool access + credits
Fairness metric	Virtual runtime	Agent virtual runtime (Eq. 3)
Weight source	Nice values (-20 to $+19$)	Credit balance $\times$ priority
Preemption	Time-slice based	Step-boundary based
State persistence	None (volatile)	Memoization store (durable)
Data structure	Red-black tree	General balanced tree (`gb_trees`)
Evaluation	N/A	6-dimensional quality space
Oversight	N/A	Three-level oversight modes

Table 5: Comparison between Linux CFS and the Agent Scheduler.

A critical difference is *preemption granularity*. Linux CFS can preempt a process at any point via timer interrupts. Agent scheduling cannot interrupt an LLM call mid-generation; preemption occurs only at *step boundaries*—between tool calls, between LLM invocations, or at explicit checkpoints. This coarser granularity means scheduling fairness is achieved over longer time horizons, similar to cooperative multitasking (Tanenbaum and Bos, 2014).

10.2 Why Elixir/OTP

The choice of Elixir/OTP as the implementation language is deliberate:

1. **Supervision trees** are a first-class language feature, not a library add-on. The `use Supervisor` and `use DynamicSupervisor` macros generate the correct callback structure.

2. **Lightweight processes.** The BEAM VM supports millions of concurrent processes with per-process garbage collection and preemptive scheduling. Each agent runs as its own process at essentially zero overhead.

3. **Immutability by default.** All data structures are immutable. State transitions in GenServer produce new state values, making reasoning about state changes straightforward.

4. **Pattern matching.** State machine transitions are encoded as function clauses with pattern-matched arguments, providing exhaustiveness checking and clear control flow.

5. **Hot code reloading.** The BEAM supports live code upgrades, enabling agent system updates without downtime.

6. **Telemetry built-in.** The `:telemetry` library is an Erlang ecosystem standard, providing structured observability.

7. **The pipe operator.** While syntactic, the `|>` operator encourages pipeline-oriented thinking that aligns with the architectural design.

10.3 Comparison to Existing Agent Frameworks

Our framework differs from existing multi-agent systems in several ways:

Feature	**This Work**	**AutoGen**	**LangChain**
Supervision	OTP trees	None	None
Crash recovery	Memo store + restart	Manual	Manual
Scheduling	Credit-weighted CFS	Round-robin	Sequential
Streaming	Event pub/sub	Message passing	Callbacks
Evaluation	6-dim + EWMA	None	None
Concurrency	BEAM processes	Threading	Async/await

Table 6: Feature comparison with existing agent frameworks.

Traditional multi-agent systems (MAS) (Wooldridge, 2009; Dorri et al., 2018) focus on communication protocols and negotiation strategies. Our framework takes an operating systems perspective: agents are managed entities whose lifecycle, resource usage, and quality are controlled by system-level primitives. The Agent-Hero marketplace adds an economic dimension (credits, contracts, reputation) absent from most MAS frameworks.

10.4 Relationship to Durable Execution Platforms

The Inngest model of durable execution (Inngest, 2024) is increasingly adopted in production. Our formalization (Theorem 6.4) provides the theoretical foundation for understanding why memoization-based retry is correct. The key distinction between "logical determinism" (same inputs yield same memoized output) and "physical nondeterminism" (LLM calls are stochastic) is crucial for agent systems: the memo store captures specific LLM outputs, and replays use memoized results rather than re-invoking the LLM.

In the Agent-Hero production system, durable execution is implemented via Inngest (a TypeScript/Node.js durable execution engine). The Elixir reference implementation demonstrates the same principles in a functional programming context where immutability and supervision are language-level features rather than library concerns.

10.5 Production System Mapping

The reference implementation maps to the Agent-Hero production architecture:

Elixir Module	**Production Component**
`AgentScheduler` (Application)	Next.js application with Inngest
`AgentScheduler.Agent` (GenServer)	Agent execution via Vercel AI SDK
`AgentScheduler.Scheduler`	Inngest function queue + credit service
`AgentScheduler.Pipeline`	Agent-Testing-Framework 5-phase pipeline
`AgentScheduler.Supervisor`	Inngest retry + Vercel deployment
`AgentScheduler.Evaluator`	Evaluation scoring in Supabase
Registry	Supabase agents table + realtime

Table 7: Mapping from Elixir reference to Agent-Hero production system.

10.6 Limitations and Future Work

1. **Resource heterogeneity.** We treat credits as a single resource, but agents consume heterogeneous resources (different LLM providers, tool APIs, compute). Part II will develop a multi-resource scheduling model based on dominant resource fairness (Ghodsi et al., 2011).

2. **Dynamic priority.** Our priority model is static (contracted vs. marketplace). Part II will introduce dynamic priority aging and feedback-driven priority adjustment.

3. **Distributed scheduling.** Our model assumes a single scheduler. Part III will extend to distributed scheduling across multiple AIOS instances.

4. **Empirical evaluation.** Our case studies are qualitative. We plan large-scale empirical evaluation on Agent-Hero production data.

5. **Persistent memoization.** The current memo store is in-process memory, lost on supervisor termination. Production systems need persistent memoization (e.g., backed by Supabase/Postgres).

11 Conclusion

We have presented the design and implementation of an AI agent scheduler built on functional programming principles and Erlang/OTP's process management model. The key contributions are:

1. **Compositional design.** Agents compose through typed data flows and streaming pipelines. The pipe operator is not just syntax—it reflects the architectural principle that agent orchestrations are chains of transformations.

2. **Supervision for fault tolerance.** OTP supervision trees provide hierarchical, configurable crash recovery. The `rest_for_one` strategy at the application level and `one_for_one` at the agent pool level match the dependency structure of the system.

3. **Durable execution.** Memoization-based step functions guarantee idempotent replay under crash-restart (Theorem 6.4), eliminating wasted computation in long-running agent tasks.

4. **Pattern-matched state machines.** Agent lifecycle transitions are enforced by Elixir pattern matching, making invalid state transitions structurally impossible rather than checked at runtime.

5. **6-dimensional evaluation with convergent reputation.** The weighted evaluation framework provides composable quality measurement, and EWMA reputation provably converges (Theorem 8.4).

6. **Complete reference implementation.** The Elixir codebase (`lib/agent_scheduler/`) demonstrates that these principles work in practice, with production-grade supervision, telemetry, and process management.

This work establishes the "process scheduling" layer of the AI Operating System. Subsequent papers in this series address the memory hierarchy (Part II: agent memory and context management), the filesystem (Part III: knowledge storage and retrieval as a typed filesystem), the network stack (Part IV: inter-agent communication protocols), and the shell (Part V: the human-agent interface).

The central message is that **agent orchestration is process management**. Four decades of operating systems research—and three decades of Erlang/OTP production deployment—provide a rich, battle-tested foundation for building reliable, fair, and efficient AI agent systems. The functional programming model ensures this foundation is compositional: agents compose like functions, pipelines compose like pipes, supervision composes like trees, and evaluation composes like weighted sums.

Acknowledgments

We thank the YonedaAI Research Collective for discussions on compositional approaches to agent systems, and the Agent-Hero development team for access to production system architecture and execution data.

Source Code

The complete reference implementation is available at:

`agent-os/src/agent_scheduler/`

Key source files referenced in this paper:

File	Section
`lib/agent_scheduler.ex`	Application supervisor (Sec. 7)
`lib/agent_scheduler/agent.ex`	Agent GenServer + state machine (Sec. 3, 6)
`lib/agent_scheduler/scheduler.ex`	Credit-weighted scheduler (Sec. 4)
`lib/agent_scheduler/pipeline.ex`	Streaming pipeline (Sec. 5)
`lib/agent_scheduler/supervisor.ex`	DynamicSupervisor (Sec. 7)
`lib/agent_scheduler/evaluator.ex`	6-dim evaluation + EWMA (Sec. 8)
`mix.exs`	Project configuration and dependencies

References

Armstrong, J. (2003). Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm.

Bovet, D. P. and Cesati, M. (2005). *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition.

Chase, H. et al. (2024). LangChain: Building applications with LLMs through composability. https://github.com/langchain-ai/langchain.

Dorri, A., Kanhere, S. S., and Jurdak, R. (2018). Multi-agent systems: A survey. *IEEE Access*, 6:28573–28593.

Ghodsi, A., Zaharia, M., Hindman, B., Konwinski, A., Shenker, S., and Stoica, I. (2011). Dominant resource fairness: Fair allocation of multiple resource types. In *NSDI*, pages 323–336.

Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429.

Hong, S., Zhuge, M., Chen, J., Zheng, X., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR 2024*.

Inngest, Inc. (2024). Inngest: Durable execution for modern applications. https://www.inngest.com/docs.

Jennings, N. R. and Wooldridge, M. J. (1998). Applications of intelligent agents. In *Agent Technology: Foundations, Applications, and Markets*, pages 3–28.

Liu, C. L. and Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61.

Logan, D. and Theodorou, A. (2023). Elixir/OTP for fault-tolerant agent systems. In *Proceedings of the Workshop on Programming Multi-Agent Systems*, pages 45–58.

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley, 3rd edition.

Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *USENIX ATC*, pages 305–319.

Pabla, C. S. (2009). Completely fair scheduler. *Linux Journal*, 2009(184).

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *UIST 2023*, pages 1–22.

Poettering, L. (2010). systemd: A system and service manager for Linux. https://www.freedesktop.org/wiki/Software/systemd/.

Qin, Y., Liang, S., Ye, Y., et al. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR 2024*.

Reed, S., Zolna, K., Parisotto, E., et al. (2022). A generalist agent. *Transactions on Machine Learning Research*.

Shinn, N., Cassano, F., Gopinath, A., Sheshadri, K. R., Jiang, P., Balasubramanian, K., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS 2023*.

Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*. Wiley, 10th edition.

Tanenbaum, A. S. and Bos, H. (2014). *Modern Operating Systems*. Pearson, 4th edition.

Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6).

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., et al. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Xi, Z., Chen, W., Guo, X., et al. (2023). The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*.

A Complete Agent State Transition Table

From State	To State	Trigger	Guard / Pattern Match
`:pending`	`:running`	`assign_job/2`	Agent must be in `:pending`
`:pending`	`:cancelled`	`cancel/1`	Any cancellable state
`:running`	`:waiting_approval`	`request_approval/2`	Oversight mode dispatch
`:running`	`:checkpointed`	`checkpoint/1`	Agent must be `:running`
`:running`	`:completed`	`complete/2`	All steps done
`:running`	`:failed`	Step raises exception	`rescue` in execute_step
`:waiting_approval`	`:running`	`respond_approval(:approve)`	Must be `:waiting_approval`
`:waiting_approval`	`:pending`	`respond_approval(:reject)`	Retries $<$ max_retries
`:waiting_approval`	`:failed`	`respond_approval(:reject)`	Retries $\geq$ max_retries
`:checkpointed`	`:running`	`execute_step/3`	Next step resumes

Table 8: Complete state transition table. Each row corresponds to a pattern-matched `handle_call` or `handle_cast` clause in `agent.ex`.

B Module Dependency Graph

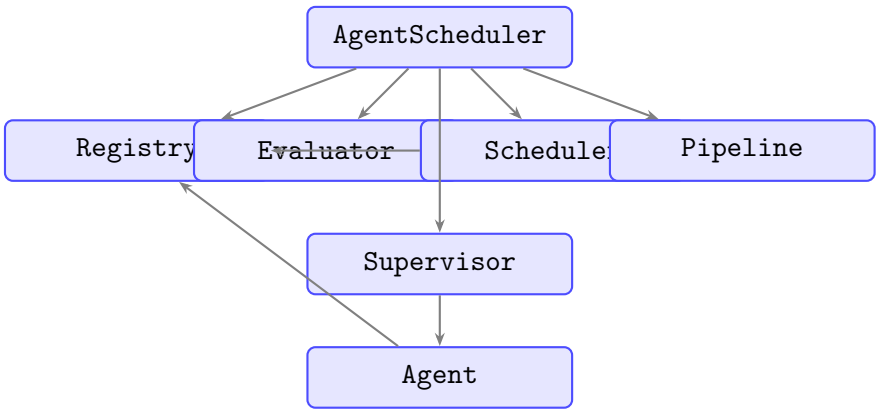

Figure 5: Module dependency graph. Arrows point from dependant to dependency. The Application module supervises all top-level components; the DynamicSupervisor manages Agent processes.

C Mix Project Configuration

```
defp deps do
  [
    {:telemetry, "~> 1.2"},
    {:jason, "~> 1.4"},
    {:ex_doc, "~> 0.31", only: :dev, runtime: false}
  ]
end
```

Listing 22: Project dependencies (`mix.exs`)

The minimal dependency footprint (only `telemetry` and `jason` at runtime) reflects the design philosophy: the BEAM VM and OTP standard library provide everything needed for supervision, concurrency, process management, and balanced tree data structures. No external scheduling libraries, state machine libraries, or pub/sub frameworks are required.